

Čtení a zápis souborů je v Pythonu o něco jednodušší než v ostatních jazycích. Existují dva základní způsoby, jak se soubory pracovat.

Čtení souborů

První způsob staví na předchozí látce a používá *try-except* blok:

try:

```
# otevřít soubor v režimu "rt" (Read Text = čtení textu)
file = open("file.txt", "rt", encoding="utf-8")

# vypsání celého souboru jako seznam řádek
print(file.readlines())
# [' řádka 1 \n', ' řádka 2 \n']

# reset čtení na začátek souboru
file.seek(0)
# vypsání celého souboru jako seznam řádek bez whitespace okrajů
print([line.strip() for line in file.readlines()])
# ['řádka 1', 'řádka 2']

file.seek(0)
# vypsání celého souboru řádku po řádku
for line in file:
    print(line.strip())

file.close()
except FileNotFoundError:
    print("Soubor nenalezen!")
    exit()
```

Používá se funkce *open*, která otevře soubor na zadané cestě (relativní nebo absolutní) v zadaném režimu a kódování.

Režimy:

- „rt“, „rb“ – read text, read binary – čtení textu/binárních dat
- „wt“, „wb“ – write text, write binary – zápis textu/binárních dat
- „xt“, „xb“ – stejné jako write ale vytvoří soubor na dané cestě, pokud neexistuje
- „rt+“ apod. – umožňuje čtení i zápis zároveň

Kódování:

- Pokud není vyplněné, bere se základní kódování operačního systému. Může být problém číst na Windows soubor vytvořený na systému Linux (výchozí kódování se neshodují).
- Nejčastěji se používá kódování UTF-8, ASCII nebo Unicode.
- Binární soubory samozřejmě žádné znakové kódování nepoužívají (pokus o otevření skončí chybou).

V ukázce výše jsou zobrazeny tři základní způsoby čtení souboru po řádkách. *Readlines* načte celý soubor do seznamu řádek, *readline* načte jednu řádku a iterátor (*for* cyklus) nad souborem také čte jednu řádku.

Důležitá metoda na složitější procházení souborů je metoda *seek*, která přesune kurzor v souboru na zadanou pozici. Má dva parametry – první je pozice, druhý je způsob interpretace pozice.

Druhý parametr má 3 možné hodnoty: 0 = od začátku souboru, 1 = od aktuální pozice, 2 = od konce souboru (pouze v binárním módu). V textovém režimu funguje ale jen hodnota 0, kvůli kódování znaků v proměnné délce (viz dokumentace UTF-8 nebo Unicode).

```
file.seek(0) - vrátí na začátek souboru
file.seek(10) - vrátí na 11. znak od začátku souboru
file.seek(10, 1) - posune o 10 znaků dál v souboru
file.seek(0, 2) - posune na konec souboru
```

Vracet se lze pouze v binárním módu čtení (první parametr záporný).

Další způsob je čtení po počtu znaků pomocí metody *read*, která přebírá počet znaků jako parametr.

```
print(file.read(25))
file.seek(0)
# -1 = až do konce souboru
print(file.read(-1))
```

Zápis do souboru

Zápis textu může vypadat následovně (opět výjimkovým způsobem):

```
try:
    # otevřít soubor v režimu "wt" (Write Text = zápis textu)
    file = open("file.txt", "wt", encoding="utf-8")

    for i in range(1, 5):
        file.write(f" řádka {i} \n")

except FileNotFoundError:
    print("Soubor nenalezen!")
    exit()
except (PermissionError, OSError) as e:
    print("Chyba zápisu!")
    print(e)
    exit()
finally:
    # při psaní je nutné volat flush, soubor by se nemusel jinak uložit celý
    file.flush()
    # a soubor zavřít
    file.close()
```

Zápis do souboru je tzv. „bufferovaný“ (dávkový). Systém má určený blok paměti, kam program zapisuje (o velikosti např. 1kB), a jakmile se buffer zaplní, systém ho celý najednou zapíše do souboru a vyprázdní pro další zápis. Proces zápisu tohoto bufferu odpovídá metodě *flush*.

Když před ukončením zápisu (a ukončením spojení se souborem) nezavoláme metodu *flush*, může se stát, že bude poslední kus zapsaného textu chybět. Buffer se v tu chvíli nezaplnil celý a operační systém tedy jeho obsah do souboru nezapsal.

Druhým (snazším) způsobem práce se soubory, je použití klíčového slova *with*:

```
with open("file.txt", "rt", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

Klíčové slovo *with* obstará výše-vedený blok *finally*. Tudíž zavolá na souboru *flush* a *close*.

Můžeme ho ale samozřejmě použít i se zpracováváním výjimek, např. *FileNotFoundError*:

```
try:
    with open("neexistujici.txt", "rt", encoding="utf-8") as file:
        for line in file:
            print(line.strip())
except FileNotFoundError:
    print("Soubor nenalezen!")
    exit()
```

Zápis funguje stejným způsobem, jen se změní režim na zápis (zde „wt“):

```
with open("file.txt", "wt", encoding="utf-8") as file:
    for i in range(1, 5):
        file.write(f" řádka {i} \n")
```

V praxi budeme používat jen způsob s *with*.

Parsování souborů

K parsování jsou důležité metody řetězců.

```
with open("file.txt", "rt", encoding="utf-8") as file:
    for line in file:
        ...
```

Odstranění bílých znaků (whitespace) z řetězce:

```
# "  řádka 4 \n"
line = line.strip()
# "řádka 4"
```

Odstranily se mezery a znak nové řádky. Případně i tabulátor a podobné znaky.

Rozdělení řetězce dle znaku – metoda *split*:

```
# řádka 1
parts = line.split(" ")
# ['řádka', '1']
```

Řetězec se rozdělil dle řetězce v prvním parametru (zde mezera).

K zápisu souborů s pevným formátem se hodí formátované řetězce:

```
file.write(f"  řádka {i} \n")
file.write(f"  řádka {self.ziskej_radku() + 1} \n")
```

Před řetězec napíšeme znak „f“ (jako „formatted“) a do řetězce můžeme vkládat složené závorky s výrazy uvnitř – hodnota proměnné, volání funkce, výsledek operace apod.

Knihovny parsování

Funkcionalita parsování je u určitých formátů souborů připravená v knihovnách. Hlavním příkladem jsou zabudované knihovny *csv* a *json* k parsování stejnojmenných formátů.

```
import csv

with open("file.csv", encoding="utf-8") as file:
    # přeskočit hlavičku
    file.readline()

    # vytvořit parser
    reader = csv.reader(file, delimiter=";")

    # iterace řádek přes parser
    for row in reader:
        print(row)

# ['1', 'pepa', '25'] - jedna řádka jako seznam řetězců (volání strip a split)
# ['2', 'josé', '54']
```

K těmto účelům se často hodí získaná data uložit do nějaké instance třídy. Python umožňuje rychlou definici datových tříd pomocí zabudované knihovny *dataclasses*.

Třidu označíme anotací *dataclass* a nemusíme již definovat konstruktor. Stačí zapsat atributy přímo do třídy (podobně jako u obyčejných tříd statické atributy), a tím se vytvoří instanční atributy.

```
from dataclasses import dataclass

@dataclass
class DataRow:
    id: int
    name: str
    value: int

    def __str__(self) -> str:
        return f"[{self.id}]: {self.name} - {self.value}"
```

A můžeme je snadno vytvářet ze záznamů ze souboru:

```
# iterace řádek přes parser
for row in reader:
    print(row)

    id = int(row[0])
    name = row[1]
    value = int(row[2])
    rows.append(DataRow(id, name, value))
```